Healthcare Analysis using Big Data Analytics

A MIMIC-IV Demo Pipeline on Hadoop, Hive, Spark and MLlib

Project Understanding & Explanation Document

---

1. Summary in One Paragraph

This project builds a complete, working big-data pipeline for hospital data. Raw clinical CSV files are loaded into the Hadoop Distributed File System (HDFS), cleaned with Apache Spark, queried for disease and hospital-utilization statistics with Apache Hive, analysed for ICU outcomes with Spark SQL, turned into machine-learning features with Spark MLlib, and used to train and evaluate a 30-day hospital-readmission prediction model. The whole stack runs in Docker containers on one laptop. Every stage saves an output (a cleaned table, a query result, a model or a chart) that the next stage uses. The data is the MIMIC-IV Clinical Database Demo, a small, free, de-identified set of real ICU records released for teaching.

2. Project Objectives

The proposal set four goals; the pipeline delivers one result for each:

- Disease statistics: which diagnoses are most common and how they differ by gender (Hive).
- Hospital utilization: admissions, average length of stay and readmission rate per admission type (Hive).
- ICU patient analysis: vital signs in the first 24 hours of an ICU stay, linked to outcomes (Spark SQL).
- Readmission prediction: a model that predicts whether a patient returns within 30 days of discharge (Spark MLlib).

3. The Dataset: MIMIC-IV Clinical Database Demo

MIMIC-IV is a well-known, de-identified ICU database used worldwide in healthcare research. The full database covers over 60,000 patients and requires data-use training to access.

The Demo version used here is an open-access subset of about 100 patients and 275 hospital admissions, released so students can build and test a pipeline without a data-use agreement. Three properties shape the whole project:

- Two groups of tables: hosp/ (admissions, patients, diagnoses_icd, d_icd_diagnoses) and icu/ (icustays, chartevents).
- All dates are shifted per patient for privacy. Calendar dates are meaningless, but gaps between two dates for the same patient (such as length of stay) are accurate.
- With only about 100 patients, the number of readmission cases is small (50). This is the most important fact to state up front when discussing the model results (Section 7).

What this means: the project shows that the pipeline is correctly engineered and would scale to the full database with the same code. It does not claim production-grade prediction accuracy, and it says so openly.

4. System Architecture: Each Tool and Why It Is There

Every component has a specific job. Knowing why each exists, not just its name, is the key to explaining the project with confidence.

Component	Role in this project
Hadoop HDFS	Distributed file storage. The raw CSVs and every cleaned table live here as files.
YARN	Cluster resource manager. Hive is configured to run its heavier aggregate queries as MapReduce jobs scheduled by YARN (small queries can fall back to local execution).
Apache Hive	A SQL interface over the data. Disease and utilization questions are written as ordinary SQL against external tables that point at the cleaned HDFS files.
Apache Spark	The main workhorse: cleaning, typing, the readmission label, ICU vitals, feature building, and model training, all as PySpark scripts.
Spark MLlib	Spark's machine-learning library: Logistic Regression, Random Forest and Gradient-Boosted Trees.
PostgreSQL	Stores Hive's metastore, the catalogue of table names, columns and file locations.
Docker Compose	Starts all of the above as separate containers on a private network with one command, and resets cleanly.
Python (Matplotlib/Seaborn)	Runs on the host machine to turn small result files into the final charts.

Data flow in one line: Raw CSVs, then HDFS, then Spark (clean), then HDFS Parquet, then two branches (Hive SQL analytics, and Spark ICU analysis plus ML), then small result files, then Python charts on the host.

5. Environment Setup: the Docker Cluster

Nine containers are defined in docker-compose.yml. They share a private Docker network, so they reach each other by name (for example, Spark reads from hdfs://namenode:8020).

Container	Purpose	Port(s)
namenode	HDFS master: tracks where every file's blocks live	9870 (UI), 8020
datanode	HDFS worker: stores the file blocks	internal
resourcemanager	YARN master: schedules jobs	8088 (UI)
nodemanager	YARN worker: runs job tasks	internal
hive-metastore-postgresql	Postgres database holding Hive's table catalogue	internal
hive-metastore	Hive's metadata service, linking SQL to HDFS file paths	9083
hiveserver2	Accepts SQL connections (beeline) and runs queries	10000
spark-master	Spark master; PySpark scripts are submitted from here	8080 (UI), 7077
spark-worker	Executes the distributed Spark tasks	8081

Setup lessons worth mentioning (they show depth of understanding):

- Hive must be told explicitly to use Postgres. Without the connection settings it silently falls back to its embedded Derby database and the schema never initialises correctly.
- Every Hadoop-based container needs fs.defaultFS set to hdfs://namenode:8020. Otherwise a container points at itself instead of the real NameNode and fails with 'connection refused'.
- The Hive metastore schema must be created once with schematool -initSchema before the metastore service can start.

- Docker's internal DNS can briefly fail to resolve a sibling container right after a restart. It clears up within a few retries.
- The Spark image is based on Alpine Linux, which has no prebuilt numpy/pandas packages, so they had to be compiled from source. Matplotlib was not installed there; plotting is done on the host instead.

6. The Pipeline, Step by Step

Step 1: Load raw data into HDFS

The six raw CSV files (admissions, patients, diagnoses_icd, d_icd_diagnoses, icustays, chartevents) are copied into HDFS under /data/mimic/raw/hosp/ and /data/mimic/raw/icu/. From here on, every step reads from HDFS only.

Step 2: Ingest and clean (01_ingest_clean.py)

Raw files are first read as plain text, then each column is cast to its correct type explicitly. This avoids a common Spark trap where an ID column or an ICD code (like '0010') is wrongly read as a number.

- admissions: keep only rows where discharge is after admission; compute los_days (length of stay in days); remove duplicate admission IDs.
- patients, diagnoses, d_icd_diagnoses, icustays: de-duplicated on their natural key.
- chartevents (the largest table): filtered straight away to the vital-sign item codes used later (heart rate, respiratory rate, SpO2, blood pressure, temperature).

Each cleaned table is written back to HDFS as Parquet, a compressed, typed, column-based format that is much faster to read than CSV. A cleaning_report.md records rows dropped per table, which feeds the written report.

Step 3: Readmission label (02_label.py)

This is the most important judgement call in the project, because the label is what the model learns to predict.

Definition: an admission gets readmit_30d = 1 if the same patient has another admission that starts within 30 days of this discharge and is not a planned (elective) admission.

- 30 days is the standard window used in US (CMS) readmission policy and research.
- Patients who died during the admission cannot be readmitted, so they are labelled 0 and excluded from the modelling cohort in Step 6.
- A patient's last recorded admission might have been followed by a readmission after the data ends. This cannot be known, so it is labelled 0 and stated as a limitation.

Technically this uses a Spark window function, lead(), which looks at each patient's admissions in date order and reads the next admission's start date and type.

Result: among 260 surviving admissions, 210 were not readmitted and 50 were readmitted within 30 days, about 19%. That is a realistic rate for hospital data.

Step 4: Hive analytics, disease and utilization

Five HiveQL queries run on external Hive tables that simply point at the Parquet files in HDFS. Hive does not copy the data; it reads the same files through SQL. This shows using the right tool for the job: once data is clean, business questions are naturally SQL questions.

- Top diagnoses: joins diagnoses_icd with d_icd_diagnoses (the lookup that turns an ICD code into a name) and counts admissions.
- Diagnoses by gender: the same join, split by gender.
- Utilization by admission type: admissions, average length of stay and readmission rate.
- ICU stays by care unit: number of stays and average ICU length of stay.
- Admissions by month of year: a seasonal pattern (month is meaningful after date shifting; the year is not).

A subtlety to mention: the diagnosis join must use both icd_code and icd_version, because MIMIC-IV mixes ICD-9 and ICD-10 and the same code text can mean different diagnoses in each.

Step 5: ICU analysis with Spark SQL (04_icu_analysis.py)

- For each ICU stay, vital signs from the first 24 hours are aggregated (average, minimum, maximum).
- A plausibility filter drops sensor errors (for example, heart rate outside 20 to 250 beats per minute).
- The vitals are joined with admission type, in-hospital death and patient age to compare ICU length of stay and mortality by care unit, admission type and age band.

A per-admission ICU summary is also saved and used as features in Step 6.

Step 6: Feature engineering (05_features.py)

Each admission becomes a numeric feature vector. The key correctness rule is to avoid data leakage: every feature must have been knowable at or before discharge. Nothing from after discharge, and nothing derived from the readmission itself, is allowed.

Feature	What it captures
age, gender	Basic demographics
admission_type, insurance	Context of the admission
los_days	Length of stay
num_prior_admissions	How many times the patient was admitted before this one
num_diagnoses	Number of diagnoses recorded
had_icu_stay, icu_los_days	Whether and how long the patient was in ICU
avg heart rate, resp rate, SpO2 (first 24h)	ICU vitals from Step 5

Categorical features are one-hot encoded. Missing vitals are filled with the median. Features are scaled. Race was deliberately left out: with about 200 rows and many rare categories it would mostly add noise, and using it properly needs a fairness analysis of its own.

The train/test split is done by patient, so the same patient never appears in both sets (otherwise the model could appear to do well by recognising someone it already saw). It is also stratified, so the rare positive cases are spread across both sets.

Step 7: Model training and evaluation (06_train_eval.py)

Three models are trained and compared, from simple to flexible:

1. Logistic Regression: the baseline; simple, fast, interpretable.
2. Random Forest: many decision trees voting; captures non-linear patterns.
3. Gradient-Boosted Trees: trees built one after another, each fixing the last one's errors; often strongest on tabular data but easier to overfit on small data.

Because positives are rare, class weights make mistakes on the 'readmitted' class count more. A model that always says 'no readmission' would already be about 80% accurate and useless, so accuracy is not used. Instead the evaluation reports:

- AUC-ROC and AUC-PR, standard metrics for imbalanced problems.
- The confusion matrix as raw counts, not just percentages, because with very few positive test cases a percentage can mislead.
- Precision, recall and F1 for the readmitted class.

Step 8: Visualization (07_visualize.py)

Runs on the host machine, not in Docker. It reads the small result files from earlier steps and produces nine charts: top diagnoses, admissions by month, readmission rate by admission type, ICU length of stay by care unit, ICU length-of-stay distribution, ICU stay by age band, the ROC curve, the confusion matrix and feature importance.

7. Results: What the Pipeline Found

7.1 Label balance (surviving admissions)

readmit_30d	Count	Meaning
0	210	Not readmitted within 30 days
1	50	Readmitted within 30 days

7.2 Model comparison on the test set

Model	Test AUC-ROC	Train AUC-ROC	Precision	Recall	F1
Logistic Regression	0.466	0.793	0.167	0.176	0.171
Random Forest	0.428	0.919	0.071	0.059	0.065
Gradient-Boosted Trees	0.449	0.991	0.214	0.176	0.194

How to interpret this honestly (the most important thing to tell the panel): all three test AUCs are at or below 0.5, which is what random guessing gives. At the same time, Random Forest and GBT score very high on the training data (0.92 and 0.99). That combination is the classic sign of overfitting: with only about 33 positive training cases, the tree models memorised patterns specific to those patients instead of learning something general. This was expected and was flagged in the project plan before training. It reflects the tiny demo dataset, not an error in the pipeline. On the full MIMIC-IV database the same code would have hundreds of times more positive cases and could be expected to give a genuinely useful model.

7.3 Feature importance (Random Forest)

Even though predictive power is limited by data size, the ranking is clinically sensible. The most-used features were average heart rate in the first 24 ICU hours, age, gender, number of diagnoses and SpO2, all plausible readmission risk factors.

8. Architecture Diagram (to draw on a slide)

Draw this left to right:

Raw CSV files > HDFS (raw) > Spark (clean and type) > HDFS (cleaned Parquet)

From the cleaned Parquet layer, two parallel branches:

- Branch A: Hive external tables, then HiveQL queries (disease and utilization), then small result files.
- Branch B: Spark readmission label, ICU analysis, feature engineering, MLlib training and evaluation, then a saved model, metrics and small result files.

Both branches feed Python (Matplotlib/Seaborn) on the host, which produces the report figures. Draw YARN as a box around Spark and Hive, as the resource manager.

9. Limitations (say these before you are asked)

- Tiny sample (about 100 patients, 50 readmission cases). The demo proves the pipeline; it cannot give a clinically usable model.
- The models overfit, shown by the gap between train and test AUC. This follows from the small positive class and is not a coding error.
- Right-censoring: a patient's last recorded admission cannot be checked for a later readmission that the dataset does not contain.
- Race was excluded from the features, so the model cannot be audited for racial bias in this version.
- Spark runs on its own standalone cluster, not submitted through YARN, because the Hadoop/YARN images do not provide the Python packages MLlib needs. YARN is used by Hive's MapReduce queries.
- The YARN NodeManager container crashed at times during development. The Hive analytics still completed, so if asked, say plainly that YARN stability was not fully resolved and that small Hive queries may have run in local mode.

10. Conclusion and Future Work

The project delivers a correct, reproducible, end-to-end pipeline covering ingestion, cleaning, SQL analytics, ICU analysis, feature engineering and machine learning, on an industry-standard stack (Hadoop, Hive, Spark). The modest model results, reported openly, reflect the size of the demo dataset and not the quality of the engineering.

Next steps for the future-work slide:

- Re-run the same pipeline on the full MIMIC-IV database (60,000+ patients) with more HDFS storage and cluster nodes.
- Add clinical notes and use NLP to extract further risk signals.
- Move from batch to streaming ingestion for a more realistic hospital setting.
- Run a fairness audit once the sample is large enough to include race and ethnicity responsibly.

11. Questions a Panel May Ask, with Prepared Answers

"Why is your dataset so small?"

The full MIMIC-IV database needs a data-use agreement and training. The Demo subset is open-access so a pipeline can be built and verified quickly. The same code scales to the full data.

"Your AUC is below 0.5. Is the model broken?"

No. There are too few examples for the models to separate real signal from noise, and a score near or below 0.5 on a tiny test set is within the range of chance. The high training AUC next to the low test AUC is the signature of overfitting, which we anticipated.

"Why Random Forest and GBT rather than a neural network?"

With about 100 patients of tabular data, tree ensembles are the standard, defensible choice, and their feature importances are interpretable, which matters in medicine. A neural network would need far more data and would be harder to explain.

"How did you handle class imbalance?"

Two ways: class weights during training, so errors on the rare 'readmitted' class count more, and a stratified patient-level split, so positives appear in both train and test.

"How do you know there is no data leakage?"

Every feature is restricted to information available at or before discharge. The split is by patient, and the preprocessing pipeline is fitted on the training set only.

"What would you do with more time?"

Run on the full MIMIC-IV database, add NLP on clinical notes, and perform a fairness audit.

12. Project File Map

Path	What it is
docker/docker-compose.yml	Defines and connects all 9 containers
scripts/upload_to_hdfs.sh	Loads raw CSVs into HDFS
src/01_ingest_clean.py	Spark: clean and type raw tables, write Parquet
src/02_label.py	Spark: compute the 30-day readmission label
sql/ddl/create_tables.hql	Hive: external table definitions
sql/analytics/*.hql	Hive: the five disease and utilization queries
src/04_icu_analysis.py	Spark SQL: first-24h vitals and outcomes
src/05_features.py	Spark MLlib: features and train/test split
src/06_train_eval.py	Spark MLlib: train and evaluate LR, RF, GBT
src/07_visualize.py	Host Python: generate the nine charts
reports/figures/	The generated chart PNGs
reports/cleaning_report.md, experiment_log.md	Auto-generated evidence for the written report

---